

Software Requirements Specification (SRS)

IT Smart Queue System

1. บทนำ (Introduction)

- **ชื่อ:** IT Smart Queue System
- **วัตถุประสงค์:** ระบบเว็บแอปพลิเคชันสำหรับการบริหารจัดการคิวรับบริการและคิวแจ้งซ่อมของแผนก IT โดยระบบจะทำงานควบคู่และดึงข้อมูล(Sync) แบบอัตโนมัติจากระบบแจ้งซ่อมเดิม (BMS / MSSQL) เพื่อนำมาจัดเรียงคิวตามความเร่งด่วน จัดการสถานะการทำงานของเจ้าหน้าที่ IT รวมถึงมีระบบในการจัดการเวลาการเข้ากะ (Shift Management) เพื่อเก็บสถิติการทำงานได้อย่างมีประสิทธิภาพ

2. โครงสร้างและเทคโนโลยีที่ใช้ (Tech Stack)

- **Backend Framework:** Python (Django >= 5.1.4)
- **Database:** PostgreSQL (ผ่าน `psycopg2-binary`) สำหรับระบบคิว และเชื่อมต่อกับ MSSQL (ผ่าน `pyodbc`) สำหรับระบบเดิม (BMS)
- **Background Tasks (Scheduler):** `APScheduler` สำหรับรันงานเบื้องหลัง เช่น การ Sync ข้อมูล และระบบอัตโนมัติต่างๆ
- **Deployment:** รองรับ Containerization ด้วย Docker และ Docker Compose
- **Frontend:** HTML, CSS (Bootstrap Theme: SB Admin 2), JavaScript (Ajax/Fetch API)

3. สิทธิการใช้งานและผู้ใช้ระบบ (User Roles)

- **IT Staff / Administrator:**
 - ควบคุมและจัดการคิว (เรียกคิว, จบงาน, แทรกคิว)
 - แก้ไขรายละเอียดงานหรือเปลี่ยนสถานะ
 - เปิด-ปิดกะการทำงาน (Shift)

หมายเหตุ: ระบบตรวจสอบสิทธิ์ผู้ดูแลระบบ (Admin) จาก IP Address หรือ Hostname ของเครื่องที่ใช้งาน (เช่น เครื่องที่กำหนดชื่อไว้ หรือ IP ที่ระบุ)

- **General User / Display Screen:**

- หน้าจอสำหรับดูความคืบหน้าของคิวงาน (Dashboard) สำหรับให้ผู้มารับบริการดูสถานะคิวของตนเอง

4. ฟีเจอร์หลักของระบบ (System Features)

4.1 ระบบจัดการคิว (Queue Management)

- **การดึงข้อมูลอัตโนมัติ (Data Synchronization):** ระบบจะซิงค์ข้อมูลใบแจ้งซ่อมจากฐานข้อมูล BMS (MSSQL) ทุกๆ 20 วินาที ผ่าน Background Task นำมาสร้างเป็น `QueueItem`
- **การจัดลำดับคิว:** คิวจะถูกจัดเรียงตาม "สถานะความเร่งด่วน (Urgent)" และตาม "ลำดับเวลาที่แจ้ง (Queue Number)"
- **การจัดการสถานะคิว (Queue Statuses):**
 - `WAITING` : รอคิว
 - `ACTIVE` : กำลังดำเนินการ
 - `COORDINATING` : อยู่ระหว่างประสานงาน
 - `WAITING_PARTS` : รออะไหล่
 - `DONE` : เสร็จสิ้น
- **การเรียกคิวและการจบงาน (Call & Finish Queue):**
 - เมื่อกด "เรียกคิวถัดไป" ระบบจะเปลี่ยนสถานะคิวเดิมให้เป็นเสร็จสิ้น และดึงคิว Waiting ถัดไปมาเป็น Active พร้อมเก็บประวัติเวลาเรียกคิว
 - **Validation Rule:** ก่อนการเรียกคิวถัดไปหรือจบงาน ระบบจะเช็คไปยังระบบ BMS ว่าใบงานนั้นถูกอัปเดตสถานะเป็น "ซ่อมเสร็จ(2)" หรือ "ตรวจรับงานแล้ว(12)" หรือไม่ หากยังไม่เปลี่ยนสถานะในระบบ BMS จะไม่สามารถจบงานในระบบคิวได้
- **ระบบคิวแทรกฉุกเฉิน (Ad-hoc Queue):**
 - สามารถแทรกคิวงานเข้ามาเป็นสถานะ Active ได้ทันที โดยจะมีข้อแม้ว่าต้องไม่มีคิวแทรกอื่นๆ ที่ Active อยู่ (ป้องกันการแทรกคิวซ้ำซ้อน)

- **การแก้ไขหมายเหตุและความเร่งด่วน:** สามารถแก้ไขหมายเหตุ (Comment) ของคิว และปรับเปลี่ยนป้ายกำกับความเร่งด่วน (Urgent Toggle) ได้ทันที

4.2 ระบบจัดการกะเวลา (Shift Control Management)

- **Manual Control:** เจ้าหน้าที่สามารถกดปุ่ม "เปิดกะ" หรือ "ปิดกะ" ได้จากหน้า Dashboard ระบบจะบันทึกข้อมูลเวลาและชื่อเครื่อง(Hostname) ของผู้ดำเนินการ
- **Auto-Close Shift (ปิดกะอัตโนมัติ):** ทำงานอัตโนมัติทุกๆ 5 นาที ในช่วงเวลา 21:00 - 06:00 หากไม่มีการเปิดล่วงเวลา(OT) ระบบจะสร้างประวัติการปิดกะโดย "System (Auto)"
- **Auto-Open Shift (เปิดกะอัตโนมัติ):** ระบบจะเปิดกะให้โดยอัตโนมัติทุกวัน ในเวลา 08:00 น. หากสถานะตอนนั้นยังคงปิดอยู่

4.3 ระบบหน้าปัดและแสดงผล (Dashboard & Monitoring)

- **Real-time Counters:** แสดงตัวเลขสรุปจำนวนคิวตามสถานะ (Waiting, Active, Done ภายในเดือนนั้น, Coordinating, Waiting Parts)
- **Search & Filter:** ระบบค้นหาข้อมูลคิว (รองรับการค้นหาหลายคำพร้อมกัน) ค้นหาจากรายละเอียดอาการเสีย, ชื่อผู้แจ้ง, หรือเลขคิว
- **Pagination:** การแบ่งหน้าการแสดงผล เพื่อไม่ให้หน้าเว็บโหลดข้อมูลมากเกินไป (แสดง 5 รายการต่อหน้า)

5. โครงสร้างฐานข้อมูล (Data Models)

- **QueueItem (คิวรับบริการ):** เก็บข้อมูลหลักของคิว เช่น เลขคิว (queue_number), ชื่อผู้แจ้ง (user_name), แผนก (user_department), อาการ (issue_description), สถานะเร่งด่วน (is_urgent), หมายเลข Job BMS ที่เชื่อมโยง (linked_job_no), สถานะปัจจุบัน (status), สถานะคิวแทรก (is_adhoc)
- **QueueStatus (สถานะคิว):** เก็บ Master data ของสถานะ เช่น รอคิว, กำลังดำเนินการ, เสร็จสิ้น พร้อมกำหนดสี(Badge color) เพื่อแสดงบนเว็บ
- **ShiftClosure (ประวัติกะ):** เก็บบันทึกเวลาเปิด-ปิดกะ (`closed_at` , `opened_at`) พร้อมระบุชื่อเครื่องที่เกิดดำเนินการ (`closed_by` , `opened_by`)
- **JobsBms (ข้อมูลจากระบบเก่า):** ตารางรับข้อมูลที่ดึงมาจาก MSSQL เพื่อเก็บรายละเอียดทั้งหมดของใบงานเดิมเพื่อใช้อ้างอิงและตรวจสอบสถานะ

6. สถาปัตยกรรมและการเชื่อมต่อ (Architecture & Integration)

- **Internal API (AJAX/Fetch):** หน้า Frontend จะใช้ JavaScript เรียกใช้ API ภายใน (`urls.py`) สำหรับอัปเดตข้อมูลแบบ Asynchronous เช่น `/update-job-desc/` , `/toggle-urgent/` , `/call-next/` , `/toggle-shift-status/` เป็นต้น
- **Legacy System Integration:** มีสคริปต์ `queue_app/utils.py` (ฟังก์ชัน `sync_jobs_from_mssql`) ที่จะเข้าไปเชื่อมต่อฐานข้อมูล MSSQL เพื่อดึงงานซ่อมที่มีสถานะยังไม่เสร็จ (เช่น 1, 11, 0) เข้ามาสร้างเป็น `QueueItem` ในระบบนี้โดยอัตโนมัติ